CSCE 616 – Lab 7

Driver Code:

```
class htax_tx_driver_c extends uvm_driver #(htax_packet_c);

        parameter PORTS = `PORTS;
        parameter VC    = `VC;
        parameter WIDTH = `WIDTH;

        `uvm_component_utils(htax_tx_driver_c)

        virtual interface htax_tx_interface htax_tx_intf;

        //Constructor
        function new (string name, uvm_component parent);
                super.new(name,parent);
        endfunction : new


 //UVM build phase
 function void build_phase(uvm_phase phase);
  super.build_phase(phase);
                if(!uvm_config_db#(virtual htax_tx_interface)::get(this,"","tx_vif",htax_tx_intf))
                        `uvm_fatal("NO_TX_VIF",{"Virtual Interface needs to be set for ",
get_full_name(),".tx_vif"})
        endfunction : build_phase

        //UVM run_phase
 task run_phase(uvm_phase phase);
```

```
                rst_dut_and_signal();
        //Reset the interface signals

                forever begin
                        //Till end of test

                        seq_item_port.get_next_item(req);       //Get next pkt from sequencer

                        drive_thru_dut(req);
        //Drive pkt to DUT via interface

                        seq_item_port.item_done();                              //Sends ACK to
sequencer

                end

        endtask : run_phase


        task rst_dut_and_signal();

                @(negedge htax_tx_intf.rst_n);

                `uvm_info("TOP", "Resetting the DUT synchronously", UVM_NONE)

                @(posedge htax_tx_intf.clk)

                htax_tx_intf.tx_outport_req  = 'b0;

        htax_tx_intf.tx_vc_req      = 'b0;

        htax_tx_intf.tx_sot         = 'b0;

        htax_tx_intf.tx_release_gnt  = 'b0;

        htax_tx_intf.tx_eot         = 'b0;

        endtask : rst_dut_and_signal


        extern protected task drive_thru_dut(htax_packet_c pkt);

endclass : htax_tx_driver_c


//TO DO : COMPLETE THE DRIVER CODE BELOW

task htax_tx_driver_c::drive_thru_dut(htax_packet_c pkt);


        `uvm_info (get_type_name(), $sformatf("Input Data Packet to DUT : \n%s",
pkt.sprint()),UVM_NONE)


        //Wait for pkt.delay clock cycles
```

```
repeat (pkt.delay) @(posedge htax_tx_intf.clk);

//On next clk-posedge
//TO DO : Set htax_tx_intf.tx_outport_req in one-hot fashion from pkt.dest_port
//TO DO : Assign htax_tx_intf.tx_vc_req from pkt.vc
@(posedge htax_tx_intf.clk) begin
        htax_tx_intf.tx_outport_req = 1 << pkt.dest_port;
        htax_tx_intf.tx_vc_req = pkt.vc;
end


//TO DO : Wait till htax_tx_intf.tx_vc_gnt is received
@(posedge |htax_tx_intf.tx_vc_gnt)


//On next clk-posedge
//TO DO : Set any one bit tx_sot[vc-1:0] to 1 from the ones granted by
htax_tx_intf.tx_vc_gnt
//                                  (refer to SPEC doc for more details)
//TO DO : Drive pkt.data[0] on htax_tx_intf.tx_data
//TO DO : Reset htax_tx_intf.tx_outport_req and htax_tx_intf.tx_vc_req (to zero)
@(posedge htax_tx_intf.clk) begin
        case (htax_tx_intf.tx_vc_gnt)
                'b11,
                'b01: htax_tx_intf.tx_sot = 'b01;
                'b10: htax_tx_intf.tx_sot = 'b10;
                default: htax_tx_intf.tx_sot = 'b00;
        endcase
        htax_tx_intf.tx_data = pkt.data[0];
        htax_tx_intf.tx_outport_req = 'b0;
        htax_tx_intf.tx_vc_req = 'b0;
```

```
        end



        //TO DO : On consecutive clk-posedges drive each of the packet's pkt.data on
htax_tx_intf.tx_data

        //TO DO : Assign htax_tx_intf.tx_sot to zero after first cycle

        //TO DO : Assert htax_tx_intf.tx_release_gnt for one clock cycle when driving second last
data packet

        //TO DO : Assert htax_tx_intf.tx_eot for one clock cycle when driving last data packet

        for(int i=1; i < pkt.length; i++) begin  //TO DO : Replace XXX and YYY with appropriate
values for a packet
                @(posedge htax_tx_intf.clk) begin
                        htax_tx_intf.tx_sot = 'b00;
                        htax_tx_intf.tx_data = pkt.data[i];
                        if (i == pkt.length - 2) htax_tx_intf.tx_release_gnt = 'b1;
                        if (i == pkt.length - 1) begin
                                htax_tx_intf.tx_release_gnt = 'b0;
                                htax_tx_intf.tx_eot = 'b1;
                        end
                end
        end


        //On next clk-posedge
        //TO DO : Assign htax_tx_intf.tx_data to X and htax_tx_intf.tx_eot to zero
        @(posedge htax_tx_intf.clk) begin
                htax_tx_intf.tx_data = 'bx;
                htax_tx_intf.tx_eot = 'b0;
        end


        `uvm_info (get_type_name(), $sformatf("Ended Driving Data Packet to DUT"),
UVM_NONE)

endtask : drive_thru_dut
```

Monitor Code:

```
class htax_tx_monitor_c extends uvm_monitor;

        parameter PORTS = `PORTS;

        //Factory Registration
        `uvm_component_utils(htax_tx_monitor_c)

        //uvm_analysis_port #(htax_tx_mon_packet_c)          tx_collect_port;

        virtual interface htax_tx_interface htax_tx_intf;
        htax_tx_mon_packet_c tx_mon_packet;
        int pkt_len;

        //constructor
        function new (string name, uvm_component parent);
                super.new(name,parent);
                //tx_collect_port = new("tx_collect_port",this);
                tx_mon_packet           = new();
        endfunction : new

 //UVM build phase
 function void build_phase(uvm_phase phase);
   super.build_phase(phase);
                if(!uvm_config_db#(virtual htax_tx_interface)::get(this,"","tx_vif",htax_tx_intf))
                        `uvm_fatal("NO_TX_VIF",{"Virtual Interface needs to be set for ",
get_full_name(),".tx_vif"})
        endfunction : build_phase

        //TO DO : Complete the run_phase tasks with Hints below
```

```
	task run_phase(uvm_phase phase);
		forever begin
			pkt_len=0;

			@(posedge |htax_tx_intf.tx_vc_gnt) begin
				//TO DO : Assign tx_mon_packet.dest_port from htax_tx_intf.tx_outport_req
				case (htax_tx_intf.tx_outport_req)
					'b0000,
					'b0001: tx_mon_packet.dest_port = 'b0000;
					'b0010: tx_mon_packet.dest_port = 'b0001;
					'b0100: tx_mon_packet.dest_port = 'b0010;
					'b1000: tx_mon_packet.dest_port = 'b0011;
				endcase
			end

			@(posedge htax_tx_intf.clk);
			//TO DO : On consequtive cycles append htax_tx_intf.tx_data to tx_mon_packet.data[] till htax_tx_intf.tx_eot pulse
			while(!htax_tx_intf.tx_eot) begin // TO DO : Replace XXX with appropriate condition
				tx_mon_packet.data = new[pkt_len++] ({tx_mon_packet.data, htax_tx_intf.tx_data});
				@(posedge htax_tx_intf.clk);

			end
			tx_mon_packet.data = new[pkt_len++] ({tx_mon_packet.data, htax_tx_intf.tx_data});
			tx_mon_packet.print();
		end
	endtask : run_phase

endclass : htax_tx_monitor_c
```

HTAX Packet and HTAX TX Monitor Packet:

```
--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :   42
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[RNTST]     1
[TEST_DONE]     1
[TOP]     5
[UVMTOP]     1
[htax_tx_driver_c]    30
[simple_random_seq]     3
[simple_random_test]     1
Simulation complete via $finish(1) at time 13030 NS + 51
/opt/coe/cadence/XCELIUM/tools/methodology/UVM/CDNS-1.1d/sv/src/base/uvm_root.svh:457     $finish;
xcelium> exit
```

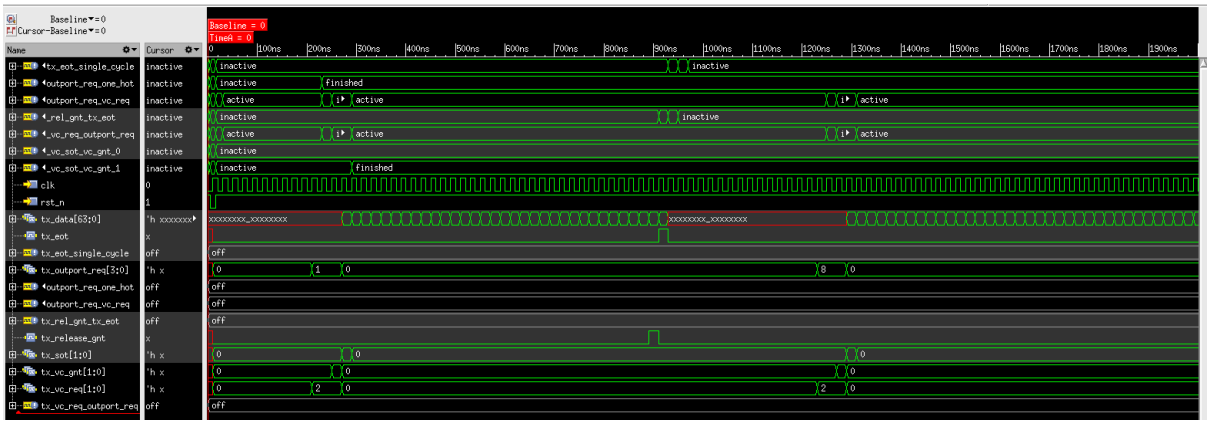
```
----------------------------------------------------------------------------------------------------------
Name                          Type          Size  Value
----------------------------------------------------------------------------------------------------------
req                           htax_packet_c -     @6393
  delay                       integral      32    'h9
  dest_port                   integral      32    'h0
  vc                          integral      2     'h2
  length                      integral      32    'h21
  data                        da(integral)  33    -
    [0]                       integral      64    'h77ccbfc7098f2f32
    [1]                       integral      64    'h14286bcac537e62b
    [2]                       integral      64    'he0423c62b3cd1e5d
    [3]                       integral      64    'hab2405b4d9c47248
    [4]                       integral      64    'h6769fbc443edfee5
    ...                       ...           ...   ...
    [28]                      integral      64    'h76a973390d7e8d48
    [29]                      integral      64    'h1a5f97156b32a728
    [30]                      integral      64    'h3ce323f6e8536e9
    [31]                      integral      64    'h133c872a769e1887
    [32]                      integral      64    'h8cf250dfc03281ba
  begin_time                  time          64    10000
  depth                       int           32    'd2
  parent sequence (name)      string        17    simple_random_seq
  parent sequence (full name) string        54    uvm_test_top.tb.tx_port[1].sequencer.simple_random_seq
  sequencer                   string        36    uvm_test_top.tb.tx_port[1].sequencer
----------------------------------------------------------------------------------------------------------

----------------------------------------------------------------
Name                  Type                  Size  Value
----------------------------------------------------------------
htax_tx_mon_packet_c  htax_tx_mon_packet_c  -     @4568
  dest_port           integral              32    'h0
  data                da(integral)          33    -
    [0]               integral              64    'h77ccbfc7098f2f32
    [1]               integral              64    'h14286bcac537e62b
    [2]               integral              64    'he0423c62b3cd1e5d
    [3]               integral              64    'hab2405b4d9c47248
    [4]               integral              64    'h6769fbc443edfee5
    ...               ...                   ...   ...
    [28]              integral              64    'h76a973390d7e8d48
    [29]              integral              64    'h1a5f97156b32a728
    [30]              integral              64    'h3ce323f6e8536e9
    [31]              integral              64    'h133c872a769e1887
    [32]              integral              64    'h8cf250dfc03281ba
----------------------------------------------------------------
UVM_INFO ../tb/htax_tx_driver_c.sv(116) @ 930000: uvm_test_top.tb.tx_port[1].tx_driver [htax_tx_driver_c] Ended Driving Data Packet to DUT
```

Output with details of the HTAX Packet / HTAX TX Monitor Packet.

Result of dest_port and data match for both.

Htax_tx_intf signals:



Output confirming no UVM_FATAL/UVM_ERROR/Assertion failures.